

LABORATORIO 13

¿Quién creó ese tópico?

*Kafka Connect, y cómo se llena un tópico sin escribir una línea de código.
Guía para hacerlo por tu cuenta, paso a paso*

Confluent Platform 8.2.0 · 3 brokers KRaft · Kafka Connect · PostgreSQL

Duración estimada · 45 minutos

NovaTech Logistics · caso de estudio

Antes de empezar

Qué vas a lograr

Éste es el laboratorio donde Kafka deja de ser algo a lo que hay que escribirle. Hasta ahora, cada mensaje que entró a un tópico lo pusiste tú, con un comando.

Hoy no vas a escribir en Kafka ni una sola vez, y el tópico se va a llenar igual. Vas a escribir una fila en una base de datos, y va a aparecer un mensaje en Kafka.

LO QUE ESTE LABORATORIO DEMUESTRA

Mover datos de un sistema a Kafka no es un programa que hay que escribir. Es una instrucción que hay que entregar.

Qué necesitas tener listo

Requisito	Cómo lo compruebas
Docker Desktop corriendo, con 6 GB de RAM	Ajustes de Docker Desktop, pestaña <i>Resources</i>
Git Bash abierto en la carpeta del lab	<code>cd Capitulo_5/lab-13-kafka-connect</code>
Los puertos 9092, 9093, 9094, 8083 , 15432 y 8090 libres	El paso 1 te avisa si no lo están
Nada más	Todo el recorrido va por <code>curl</code> . No hace falta instalar nada

POR QUÉ ESTA GUÍA USA `CURL` Y NO LOS ENVOLTORIOS DEL CURSO

Hay dos razones, y la segunda está medida.

La primera. Kafka Connect **es** una API REST y no tiene otra interfaz. No hay archivo que editar ni servicio que reiniciar. Verlo con `curl` es verlo como es, y en el servidor de SUNAT no va a haber una carpeta `connect-cli/`.

La segunda. Cuatro de los cinco envoltorios usan `python3` para dar formato a la salida, y Git Bash para Windows no lo trae. **Uno de ellos falla de una forma que te puede costar media hora**, y está desarrollado en el *para profundizar*.

El caso

El sistema que le importa a SUNAT no es Kafka. **Es la base de datos que ya estaba.** La que lleva veinte años funcionando, la que tiene los datos de verdad, la que nadie va a reemplazar este año ni el que viene.

Y el encargo siempre llega igual. *Necesitamos que lo que entra ahí también llegue a Kafka.*

La respuesta obvia es escribir un programa. Un proceso que se conecte a la base, consulte cada tanto qué hay de nuevo, y publique lo que encuentre. Suena a un par de días de trabajo, y lo es. **La primera versión.**

Lo que no suena a un par de días es todo lo que aparece después.

Cómo sabe qué filas ya publicó, si el proceso se reinicia. Qué hace si se cae a la mitad de una tanda. Qué hace si la base no responde. Cómo se reparten el trabajo dos copias sin duplicar. Quién lo monitorea y dónde están sus métricas. **Ese programa deja de ser un script y se convierte en un sistema.** Y cuando mañana el encargo se repita con otra base, o con un archivo, o con una API, hay que volver a escribirlo entero.

Y LA SEGUNDA MITAD, QUE ES LA QUE SALE MÁS CARA

Ese programa **hay que mantenerlo.** Tiene dueño, tiene despliegue, tiene versiones. Y tiene un día en que la persona que lo escribió ya no trabaja ahí.

LA METÁFORA · HOY ENTRA EL LIBRO QUE HABÍA ANTES

Kafka sigue siendo **el restaurante**. El **mozo** es el broker, el **tipo de comanda** es el tópico, los **sectores del salón** son las particiones y el **pincho** del turno es el segmento.

La pieza de hoy no es del restaurante. **Es del local que había antes.**

En el restaurante	En Kafka
El libro de reservas de la entrada , escrito a mano, que existe desde antes que las comandas y que nadie va a jubilar	La base de datos que ya estaba
La instrucción escrita , que dice «mira ese libro cada cinco segundos y copia lo que sea nuevo»	El conector
El ayudante que efectivamente lo hace, línea por línea	La task
El dedo con que marca hasta qué línea ya copió	El offset del conector

Lo que importa del ayudante, y es el corazón del laboratorio. **El ayudante no piensa**. No decide qué copiar, no interpreta, no corrige. Mira el libro, ve que hay líneas más abajo de donde tiene el dedo, y las copia a comandas. Después mueve el dedo. Y vuelve a mirar a los cinco segundos.

Nadie escribió un programa para el ayudante. **Alguien le escribió una instrucción en un papel**. Qué libro, cada cuánto, y cómo saber cuál línea es nueva.

LA REGLA QUE SALE DE AHÍ

Al ayudante no se le programa. **Se le escribe la instrucción y se le entrega.**

Y ESTO CONVIENE VERLO VENIR, PORQUE EL PASO 6 TE LO DEJA SERVIDO

El dedo solo avanza. Si alguien vuelve atrás y corrige una línea que el ayudante ya copió, el ayudante **nunca se entera**, porque su dedo ya pasó por ahí. Y si alguien arranca una hoja, tampoco.

Las cuatro piezas, con sus nombres

CONCEPTO · KAFKA CONNECT

Un servicio **aparte del clúster**, aquí el contenedor `kafka-connect`, que responde en `http://localhost:8083`. Su único trabajo es mover datos entre Kafka y sistemas externos. **No es parte del broker.**

CONCEPTO · CONECTOR

La declaración, en JSON, de **qué** conectar. No dice cómo. Dice qué base, qué tabla, con qué credenciales, cada cuánto y a qué tópico. Es la instrucción del papel. **Es todo lo que vas a escribir hoy, y no es código.**

CONCEPTO · TASK

La unidad que ejecuta el trabajo. Un conector puede tener varias, y ahí está su paralelismo. **Es el ayudante.**

CONCEPTO · WORKER

El proceso donde viven las tasks. En este laboratorio hay uno. En producción hay varios, y **si uno se cae, los demás se reparten sus tasks solos**. Es el puesto de trabajo.

Y las dos direcciones, que se nombran **desde el punto de vista de Kafka**.

Tipo	De dónde	A dónde	Cuál es hoy
Source	Un sistema externo	Kafka	El del recorrido
Sink	Kafka	Un sistema externo	El del <i>para profundizar B</i>

El tópico que todavía no existe

Qué vamos a hacer

Levantar el laboratorio y fijar el punto de partida. **La demostración entera depende de él**, así que conviene dejar los dos números a la vista.

Levantar

```
cd Capitulo_5/lab-13-kafka-connect
chmod +x bin/*.sh connect-cli/*.sh kafka-cli/*.sh
bin/start-lab.sh
```

bin/start-lab.sh — levanta los tres brokers, PostgreSQL, Kafka Connect y Kafbat UI. **Siembra cinco pedidos en la tabla** y crea vacía la tabla de destino del camino de vuelta

EL FINAL DE LO QUE VAS A VER · RECORTADO

El tópico 'novatech.lab09.pedidos' lo creará el Source connector al arrancar.

CLÚSTER NOVATECH LAB 13 OPERATIVO

Componentes:

Broker 1:	localhost:9092
Broker 2:	localhost:9093
Broker 3:	localhost:9094
Kafbat UI:	http://localhost:8090
PostgreSQL:	localhost:15432 (user: novatech, db: novatech_orders)
Kafka Connect:	http://localhost:8083 ← REST API

PostgreSQL:

5 pedidos seed creados en la tabla 'pedidos'
Tabla 'pedidos_procesados' creada vacía

Qué tópicos hay ahora

```
kafka-cli/list-topics.sh
```

kafka-cli/list-topics.sh — envoltorio del curso. Por dentro llama a `kafka-topics --list` dentro del contenedor. **Éste no usa python3** y funciona en cualquier parte

LO QUE VAS A VER

```
_connect-configs
_connect-offsets
_connect-status
novatech.lab09.pedidos.procesados
```

Cuatro tópicos, y **ninguno se llama** `novatech.lab09.pedidos`. Ése es el que va a aparecer solo.

Tópico	Qué es
<code>_connect-configs</code> , <code>_connect-offsets</code> , <code>_connect-status</code>	Son de Connect, y son de Kafka. Connect no tiene base de datos propia. Guarda sus conectores, su avance y su estado en tres tópicos del propio clúster. Por eso, si el contenedor se cae y vuelve, retoma exactamente donde iba
<code>novatech.lab09.pedidos.procesados</code>	El del camino de vuelta, que el recorrido no usa. Está en el <i>para profundizar B</i>

Y del otro lado, la tabla que sí tiene datos

```
docker exec postgres psql -U novatech -d novatech_orders \  
-c "SELECT count(*) FROM pedidos;"
```

docker exec postgres — ejecuta dentro del contenedor de PostgreSQL. Sin `-i` ni `-t`, porque la sentencia va en el propio comando y no la vas a teclear

psql -U novatech -d novatech_orders — el cliente de línea de comandos de PostgreSQL, con su usuario y su base

-c "SELECT ..." — ejecuta esa sentencia y sale. Sin `-c` abriría una sesión interactiva de la que habría que salir con `\q`

LO QUE VAS A VER

```
count  
-----  
      5  
(1 row)
```

VAS BIEN SI...

Cinco filas en la base y cero mensajes en Kafka. No existe todavía el tópico `novatech.lab09.pedidos`. Ése es el antes, y los dos números tienen que quedar anotados.

SI `PSQL` DICE `CONNECTION TO SERVER ON SOCKET ... FAILED`

Copiaste solo la línea del `psql` , sin el `docker exec postgres` de delante. **No es la base del laboratorio la que falla.** Es tu propia máquina diciendo que ahí no hay ningún PostgreSQL.

SI EL `LIST-TOPICS.SH` NO MUESTRA LOS TRES `_CONNECT-*`

Connect todavía está arrancando y aún no los ha creado. Tarda más que los brokers. Espera medio minuto y repite. Si sigue:

```
curl -s http://localhost:8083/
```

SI EL PUERTO 15432 ESTÁ OCUPADO

El laboratorio publica PostgreSQL en el **15432** y no en el 5432 estándar, justamente para no chocar con un PostgreSQL que ya tengas instalado. Si aun así está ocupado, mira quién lo tiene con `docker ps` .

La instrucción, que es un JSON

Qué vamos a hacer

Éste es el paso donde no se escribe código. Antes de enviar la instrucción hay que leerla, porque es corta y **cada línea decide algo**.

Primero, leerla

```
cat infra/connect/jdbc-source-pedidos.json
```

LO QUE VAS A VER

```
{
  "name": "novatech-source-pedidos",
  "config": {
    "connector.class": "io.confluent.connect.jdbc.JdbcSourceConnector",
    "tasks.max": "1",
    "connection.url": "jdbc:postgresql://postgres:5432/novatech_orders",
    "connection.user": "novatech",
    "connection.password": "novatech_secret",
    "table.whitelist": "pedidos",
    "mode": "incrementing",
    "incrementing.column.name": "id",
    "topic.prefix": "novatech.lab09.",
    "poll.interval.ms": "5000",
    "key.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "key.converter.schemas.enable": "false",
    "value.converter.schemas.enable": "false"
  }
}
```

Ésa es la instrucción entera. Línea por línea:

Campo	Qué decide
<code>connector.class</code>	Qué tipo de ayudante. Éste sabe hablar JDBC. Hay otros que saben S3, Elasticsearch o MongoDB. Es lo único que cambia si mañana la fuente es otra
<code>tasks.max: 1</code>	Cuántos ayudantes. Uno basta para una tabla
<code>connection.url</code>	Fíjate en el <code>postgres:5432</code> . Es el nombre del contenedor y su puerto interno . Desde tu máquina PostgreSQL está en el 15432 , pero Connect lo ve por la red de Docker
<code>table.whitelist: pedidos</code>	Qué tabla mirar. Acepta varias, separadas por coma
<code>mode: incrementing</code>	Cómo sabe qué es nuevo. Es el dedo
<code>incrementing.column.name: id</code>	Cuál es la columna del dedo
<code>topic.prefix</code>	Aquí está la respuesta al título de esta guía. El tópico no se nombra. Se arma, pegando este prefijo al nombre de la tabla, o sea <code>novatech.lab09.</code> más <code>pedidos</code>
<code>poll.interval.ms: 5000</code>	Cada cuánto vuelve a mirar
<code>value.converter</code>	En qué formato deja el mensaje. Aquí JSON. En el laboratorio 11 hubiera sido Avro

Ahora se envía

```
curl -s -w "\nHTTP %{http_code}\n" -X POST \
  -H "Content-Type: application/json" \
  --data @infra/connect/jdbc-source-pedidos.json \
  http://localhost:8083/connectors \
  | fold -s -w 88
```

-s — silencia la barra de progreso de `curl`

-w "\nHTTP %{http_code}\n" — **imprime el código HTTP al final**. Es el que dice si el conector quedó creado

-X POST a `/connectors` — crear un conector es una llamada REST y nada más

-H "Content-Type: application/json" — **sin esta cabecera Connect contesta 415**

--data @archivo — **la arroba** le dice a `curl` que el cuerpo sale de ese archivo tal cual. El JSON ya tiene la forma que Connect espera, **así que aquí no hay que escapar nada**, al revés de lo que pasaba en el laboratorio 11

| `fold -s -w 88` — corta la respuesta en líneas de 88 caracteres, cortando en espacios. Sin esto es una sola línea larguísima

LO QUE VAS A VER · EL MEDIO VA RECORTADO, PORQUE DEVUELVE LA CONFIGURACIÓN ENTERA

```
{"name":"novatech-source-pedidos","config":{"connector.class":"io.confluent.connect.jdbc
.JdbcSourceConnector","tasks.max":"1", ... ,"name":"novatech-source-pedidos"},"tasks":[]
,"type":"source"}
HTTP 201
```

Cómo se lee

`HTTP 201 Created` . El conector quedó.

Y `"tasks": []` , **vacío**. Todavía no hay ayudante. El conector quedó registrado, y el reparto de tasks ocurre un instante después. Por eso el paso 3 no es opcional.

VAS BIEN SI...

`HTTP 201` , y la respuesta te devuelve tu propia configuración con un `"type":"source"` al final. **Connect te está repitiendo lo que entendió**. Si algún campo no aparece, es que lo escribiste mal.

SI CONTESTA `HTTP 409`

Ya existe un conector con ese nombre. Casi siempre es que corriste el comando dos veces.

Mira qué hay y, si quieres empezar de nuevo, bórralo:

```
curl -s http://localhost:8083/connectors
curl -s -X DELETE -w "HTTP %{http_code}\n" \
  http://localhost:8083/connectors/novatech-source-pedidos
```

SI CONTESTA `HTTP 415`

Falta la cabecera `Content-Type: application/json` . El **415** quiere decir «no hablo ese formato», y lo dice Connect antes de mirar el contenido.

SI CONTESTA `HTTP 400` CON ALGO SOBRE EL *PAYLOAD*

Se te perdió la `@` del `--data @archivo`. **Sin la arroba, `curl` manda el texto `infra/connect/jdbc-source-pedidos.json` como si fuera el cuerpo**, en vez del contenido del archivo.

Los dos estados, que no son el mismo

Qué vamos a hacer

Preguntar por el estado del conector. Y aprender la distinción que **más se confunde en producción**.

El comando

```
curl -s http://localhost:8083/connectors/novatech-source-pedidos/status \
  | tr ',' '\n'
```

`/connectors/<nombre>/status` — el estado en vivo. Es un `GET`, así que no lleva `-X` ni cabecera `| tr ',' '\n'` — parte la respuesta en una línea por coma. Es un corte bruto y no un formateador de JSON, así que deja llaves y corchetes descolgados. Pero se lee, y no necesita nada instalado

SI LO CORRES PEGADO AL COMANDO ANTERIOR, TE VA A CONTESTAR ESTO

```
{ "error_code": 404
  "message": "No status found for connector novatech-source-pedidos" }
```

No está roto y el conector no falló. El estado se publica un instante después de aceptar la instrucción, y tú llegaste antes.

Vuelve a correr el mismo comando. Medido, la ventana dura decenas de milisegundos al recrear un conector, pero **en la primera creación después de levantar el laboratorio puede durar más**, y hacer falta dos o tres consultas. Es el único punto del recorrido donde hay que repetir un comando.

LO QUE VAS A VER CUANDO YA HAYA ESTADO

```
{ "name": "novatech-source-pedidos"
  "connector": { "state": "RUNNING"
    "worker_id": "kafka-connect:8083"
    "version": "10.9.0"
    "tasks": [ { "id": 0
      "state": "RUNNING"
      "worker_id": "kafka-connect:8083"
      "version": "10.9.0" } ]
    "type": "source" }
```

Cómo se lee

Hay dos `state`, y no son el mismo.

Cuál	Dónde está en la salida	Qué dice
<code>connector.state</code>	El <code>"state": "RUNNING"</code> que viene debajo de <code>"connector": {</code>	Si la instrucción está aceptada y vigente
<code>tasks[0].state</code>	El <code>"state": "RUNNING"</code> que viene debajo de <code>"tasks": [{</code>	Si el ayudante está trabajando

LOS DOS TIENEN QUE DECIR `RUNNING`

Un conector en `RUNNING` con su task en `FAILED` **es el caso que más caro sale**. La instrucción está ahí, se ve verde por encima, y no se está copiando nada. El *para profundizar* C lo provoca a propósito para que lo reconozcas.

VAS BIEN SI...

Puedes señalar **los dos** `RUNNING` en la salida y decir cuál es cuál. **Anota una respuesta**. ¿Cuál de los dos miraría un tablero que solo muestra «el conector está bien»?

SI EL 404 NO SE VA DESPUÉS DE VARIOS INTENTOS

Comprueba primero que el conector existe, que es otra pregunta:

```
curl -s http://localhost:8083/connectors
```

Si la lista está vacía, el `POST` del paso 2 no llegó a crear nada. Vuelve a él y mira el código HTTP.

SI EL `TASKS[0].STATE` DICE `FAILED`

En la misma respuesta hay un campo `trace` con el motivo. Léelo entero:

```
curl -s http://localhost:8083/connectors/novatech-source-pedidos/status \
  | tr '{' '\n' | grep -A2 trace
```

La causa más común aquí es que PostgreSQL todavía no aceptaba conexiones cuando el conector arrancó. **Reinicia solo la task:**

```
curl -s -X POST \
  "http://localhost:8083/connectors/novatech-source-pedidos/restart?
  includeTasks=true&onlyFailed=true"
```

El tópico que apareció solo

Qué vamos a hacer

Nadie ejecutó un comando de creación de tópicos. Vamos a mirar qué hay ahora.

```
kafka-cli/list-topics.sh
```

LO QUE VAS A VER

```
_connect-configs
_connect-offsets
_connect-status
novatech.lab09.pedidos
novatech.lab09.pedidos.procesados
```

Apareció `novatech.lab09.pedidos`. **Ésa es la respuesta al título de la guía.** Lo creó el conector, con el nombre que salió de pegar el `topic.prefix` al nombre de la tabla.

SI TODAVÍA NO APARECE, LLEGASTE ANTES QUE EL AYUDANTE

El conector tarda hasta 5 segundos en su primera pasada. Según lo rápido que vayas te vas a topar con una de estas tres, **y las tres son la misma cosa.**

Comando	Lo que contesta si aún no hay tópico
<code>list-topics.sh</code>	El tópico simplemente no está en la lista
<code>kafka-get-offsets</code>	<code>Error occurred: Could not match any topic-partitions with the specified filters</code>
<code>kafka-console-consumer</code>	Un <code>WARN ... {novatech.lab09.pedidos=UNKNOWN_TOPIC_OR_PARTITION}</code> , y después funciona igual en cuanto el tópico nace

Espera unos segundos y repite. Es el reloj del laboratorio, no un fallo.

Y no apareció vacío

CONCEPTO · PARTICIÓN

Cada uno de los pedazos en que se corta un tópico. **Éste tiene una sola**, porque el conector lo creó con los valores por defecto del broker y nadie le pidió otra cosa.

```
docker exec kafka-broker-1 kafka-get-offsets \  
  --bootstrap-server kafka-broker-1:29092 \  
  --topic novatech.lab09.pedidos --time -1
```

--time -1 — el offset **más nuevo**. Con **-2** pedirías el más antiguo

LO QUE VAS A VER

```
novatech.lab09.pedidos:0:5
```

Es **tópico:partición:offset**. Partición 0, offset 5. **Cinco mensajes**, exactamente las cinco filas que el paso 1 contó en la tabla.

El ayudante entró, encontró el libro con cinco líneas y el dedo en cero, y las copió todas.

Ahora mira lo que copió

```
docker exec kafka-broker-1 kafka-console-consumer \  
  --bootstrap-server kafka-broker-1:29092 \  
  --topic novatech.lab09.pedidos \  
  --from-beginning --max-messages 5
```

--from-beginning — empieza por el mensaje más viejo que el tópico conserva, no por lo que llegue de ahora en adelante

--max-messages 5 — **sin esto el comando no termina nunca**, porque un consumidor se queda esperando. Con el tope lee cinco y sale solo

LO QUE VAS A VER

```
{ "id":1, "cliente_id":1001, "producto":"Caja de bananos
premium", "cantidad":50, "monto":"ExLQ", "estado":"pendiente", "creado_en":1787975289752}
{ "id":2, "cliente_id":1002, "producto":"Pallet de cajas
reforzadas", "cantidad":20, "monto":"AIfNoA==", "estado":"pendiente", "creado_en":1787975289752}
{ "id":3, "cliente_id":1003, "producto":"Etiquetas RFID
x1000", "cantidad":1, "monto":"RKog", "estado":"pendiente", "creado_en":1787975289752}
{ "id":4, "cliente_id":1004, "producto":"Cinta adhesiva industrial
50m", "cantidad":100, "monto":"C3Gw", "estado":"pendiente", "creado_en":1787975289752}
{ "id":5, "cliente_id":1005, "producto":"Stretch film
500m", "cantidad":30, "monto":"G3dA", "estado":"pendiente", "creado_en":1787975289752}
Processed a total of 5 messages
```

Un JSON por línea, y **sus campos son las columnas de la tabla**. Nadie definió ese formato. Lo dedujo el conector del esquema de PostgreSQL.

El `creado_en` es la hora en milisegundos desde 1970 y **va a ser distinto en tu corrida**. Los demás campos no.

Y AHORA LO QUE SIEMPRE SE PREGUNTA · EL `"MONTO": "EXLQ"`

No está corrupto. La columna `monto` es un `NUMERIC(10,2)` de SQL, que es un decimal exacto. Y **JSON no tiene decimal exacto**. Solo tiene el número de coma flotante, que redondea.

Antes que entregarte un importe redondeado, el conector te entrega **los bytes originales en base64**. Es incómodo, y es la respuesta correcta. En un sistema tributario, un céntimo perdido por redondeo es un problema de verdad. Se puede cambiar, y el *para profundizar* D muestra qué se pierde al hacerlo.

VAS BIEN SI...

El offset dice **5**, el consumidor trajo **cinco** JSON, y los cinco `id` van del 1 al 5. **Coinciden con las cinco filas del paso 1**, y tú no escribiste ni un mensaje.

SI EL CONSUMIDOR SE QUEDA COLGADO Y NO VUELVE

Se te olvidó el `--max-messages 5`, o pediste más mensajes de los que hay. **Si pides 10 sobre un tópico que tiene 5, el consumidor espera los otros 5 para siempre**. Sal con `Ctrl+C`.

La fila nueva, y el viaje

Qué vamos a hacer

Todo lo anterior se podría explicar como «el conector hizo una carga inicial». **Este paso demuestra que el ayudante sigue mirando.**

Vas a escribir un `INSERT` en SQL. **En Kafka no vas a tocar nada.**

El comando

```
kafka-cli/insertar-pedido.sh 2001 "Pedido de la clase" 5 25000.00
```

`kafka-cli/insertar-pedido.sh` — envoltorio del curso. **A pesar de estar en `kafka-cli/`, no habla con Kafka.** Hace un `INSERT` en PostgreSQL y nada más

`2001` — el `cliente_id`

`"Pedido de la clase"` — el producto. **Elige un texto reconocible**, porque lo vas a buscar dentro de un mensaje de Kafka en un minuto

`5` — la cantidad

`25000.00` — el monto

El comando real que corre por debajo es éste. **No lo ejecutes, porque ya lo hiciste.** Está aquí para que veas que no hay nada más.

```
docker exec postgres psql -U novatech -d novatech_orders -c \  
  "INSERT INTO pedidos (cliente_id, producto, cantidad, monto, estado)  
  VALUES (2001, 'Pedido de la clase', 5, 25000.00, 'pendiente') RETURNING id;"
```

`docker exec postgres` — entra al contenedor de PostgreSQL. **En el servidor de SUNAT esta parte no existe**, porque allí `psql` está en el `PATH` y la base es local o se alcanza con `-h`

`RETURNING id` — es de PostgreSQL. **Devuelve el `id` que le tocó a la fila nueva**, sin tener que consultarlo después

LO QUE VAS A VER

```
id
----
6
(1 row)

INSERT 0 1
```

`RETURNING id` devolvió **6**. Es el `id` de la fila nueva, y por lo tanto **la línea a la que el dedo del ayudante todavía no llegó**. El `INSERT 0 1` es PostgreSQL diciendo que insertó una fila.

Y ahora se espera

Cuenta hasta cinco en voz alta, y pregunta.

```
docker exec kafka-broker-1 kafka-get-offsets \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab09.pedidos --time -1
```

LO QUE VAS A VER

```
novatech.lab09.pedidos:0:6
```

ESE 6 ES TODA LA AFIRMACIÓN DEL LABORATORIO EN UN SOLO NÚMERO

Pasó de 5 a 6. **Nadie ejecutó un productor. Nadie escribió código.** Se escribió una fila en una base de datos y apareció un mensaje en Kafka.

CUÁNTO TARDA, Y POR QUÉ NO SIEMPRE LO MISMO

En la corrida medida el offset ya había subido **a los 10 segundos** del `INSERT`. El `poll.interval.ms` son 5, así que lo normal es **entre 5 y 10 segundos**, y depende de en qué punto de sus cinco segundos estaba el ayudante cuando insertaste. **Si a ti tarda 3 o tarda 9, está bien.** La latencia de este laboratorio no la pone tu máquina. La pone un temporizador.

Y por último, el mensaje

```
docker exec kafka-broker-1 kafka-console-consumer \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab09.pedidos \
  --partition 0 --offset 5 --max-messages 1
```

`--partition 0` — obligatorio cuando se pide un offset concreto, porque los offsets son por partición

`--offset 5` — el sexto mensaje es el offset 5, porque los offsets empiezan en 0. Si insertaste más filas, súmalas

`--max-messages 1` — lee uno y sale

LO QUE VAS A VER

```
{ "id": 6, "cliente_id": 2001, "producto": "Pedido de la
clase", "cantidad": 5, "monto": "JiWg", "estado": "pendiente", "creado_en": 1787975408768 }
Processed a total of 1 messages
```

`"producto": "Pedido de la clase"` . Es el texto que escribiste en el `INSERT` , dentro de un mensaje de Kafka, sin haber tocado Kafka.

LO LOGRASTE SI...

El offset pasó de 5 a 6, y el mensaje del offset 5 trae tu propio texto.

Anota una respuesta. ¿Cuántas líneas de código escribiste hoy para que ese dato llegara a Kafka?

SI EL OFFSET NO SUBE DESPUÉS DE UN MINUTO

Mira los dos `state` del paso 3. Si la task está en `FAILED` , el ayudante murió y el `INSERT` se quedó en la base.

Y comprueba que la fila entró de verdad:

```
docker exec postgres psql -U novatech -d novatech_orders \
-tAc "SELECT max(id) FROM pedidos;"
```

SI EL MENSAJE DEL OFFSET 5 NO ES EL TUYO

Insertaste más de una fila, o repetiste el paso. El offset del último mensaje es siempre el **total menos uno**. Mira el `kafka-get-offsets` , réstale 1, y usa ese número.

El dedo, y dónde vive

Qué vamos a hacer

Buscar el dedo del ayudante. Es la pieza que hace que Connect sea tolerante a fallos, y la que explica la limitación con la que se cierra el laboratorio.

Pregúntale a Connect por dónde va

```
curl -s http://localhost:8083/connectors/novatech-source-pedidos/offsets
```

LO QUE VAS A VER

```
{"offsets":[{"partition":{"protocol":"1","table":"public.pedidos"},"offset":
{"incrementing":6}}]}
```

Y contrástalo con la fuente, que es lo que hay que hacer siempre.

```
docker exec postgres psql -U novatech -d novatech_orders \
  -tAc "SELECT max(id) FROM pedidos;"
```

-tAc — tres opciones juntas. La **t** quita los encabezados, la **A** quita el alineado en columnas, y la **c** es la sentencia. **Deja solo el número**, que es lo que quieres comparar

LO QUE VAS A VER

```
6
```

El **incrementing** del conector y el **max(id)** de la tabla dicen lo mismo. Ése es el dedo. Si tumbas Connect y lo levantas, retoma en el 7, y por eso no republica lo que ya publicó ni necesita archivos locales.

Y dónde vive el dedo

```
docker exec kafka-broker-1 kafka-get-offsets \
  --bootstrap-server kafka-broker-1:29092 --topic _connect-offsets --time -1 \
  | awk -F: '$3>0'
```

| awk -F: '\$3>0' — el tópico tiene 25 particiones y casi todas están vacías. **Esto deja solo las que tienen algo**. El **-F:** corta por los dos puntos y **\$3** es el offset

```
_connect-offsets:20:2
```

De las 25 particiones solo una tiene datos, porque **los registros de un conector van todos a la misma por su clave**. Y tiene 2 registros.

EL DEDO VIVE EN UN TÓPICO DE KAFKA, Y ESO LO EXPLICA TODO

Connect no tiene base de datos ni archivos locales. **Su avance está en `_connect-offsets`, replicado como cualquier otro tópico**. Por eso, si el contenedor muere, otro *worker* puede retomar exactamente donde iba. Y por eso, si borras el conector y lo vuelves a crear con el mismo nombre, **no vuelve a copiar lo que ya copió**.

Y AQUÍ ESTÁ LA LIMITACIÓN, QUE ES LO QUE HAY QUE LLEVARSE

El dedo solo avanza. El modo `incrementing` pregunta por filas con `id` mayor que el último visto, y nada más.

Si alguien hace un `UPDATE` sobre la fila 3, **el conector no se entera**. Si alguien hace un `DELETE`, tampoco. **Este conector copia altas, no cambios**. Para capturar cambios hace falta otro modo, que es `timestamp`, o una herramienta de otra familia, que es la captura de cambios desde el registro de transacciones.

LO LOGRASTE SI...

El `incrementing` del conector y el `max(id)` de la tabla te dan el mismo número, y sabes decir en qué tópico vive ese número.

Anota una respuesta. Si mañana alguien corrige el importe de un pedido ya publicado, ¿qué llega a Kafka?

SI EL `AWK` NO DEVUELVE NINGUNA LÍNEA

Todas las particiones están en 0, o sea que el conector todavía no ha guardado su avance. **Espera una pasada más** y repite. Si sigue vacío después de un minuto, mira los dos `state` del paso 3.

Lo que aprendiste

1 Mover datos a Kafka es una instrucción, no un programa.

Lo que escribiste hoy es un JSON de quince líneas, y se entregó con un `POST`. Y si mañana la fuente es S3 o Elasticsearch, **lo único que cambia es el `connector.class`**. Todo lo demás, o sea los reintentos, el reparto y el estado, ya está resuelto.

2 Hay dos `state` y hay que mirar los dos.

Un conector en `RUNNING` con su task en `FAILED` es el caso que más caro sale, porque se ve verde por encima y no se está copiando nada. Cuando alguien te diga «el conector está arriba», la pregunta correcta es cuál de los dos está mirando.

3 Connect guarda su estado en Kafka, no en disco.

Sus conectores, su avance y su estado viven en `_connect-configs`, `_connect-offsets` y `_connect-status`. Por eso retoma donde iba después de una caída, y por eso recrear un conector con el mismo nombre no vuelve a copiar lo ya copiado.

4 El dedo solo avanza, y eso decide para qué sirve.

El modo `incrementing` copia altas. Un `UPDATE` o un `DELETE` sobre una fila ya publicada no llegan nunca. Antes de proponer este conector para un caso de SUNAT, la primera pregunta es si los datos de origen se corrigen después de escritos.

Para profundizar

A · Qué sabe hacer este Connect

```
curl -s http://localhost:8083/  
curl -s http://localhost:8083/connector-plugins | tr '{' '\n' | grep -oE '"class":'  
[^\"]*"'
```

LO QUE SALE DEL PRIMERO

```
{"version":"8.2.0-  
ccs","commit":"62a39a849020cb35fe97d7dbaa83962d8bde94b5","kafka_cluster_id":""}
```

OJO CON EL `KAFKA_CLUSTER_ID`, QUE VIENE VACÍO

No está roto y no es tu clúster mal configurado. En esta versión ese campo no se llena. Es un buen ejemplo de por qué las salidas se miran y no se suponen.

LO QUE SALE DEL SEGUNDO · LOS CINCO PLUGINS DE ESTA IMAGEN

```
"class":"io.confluent.connect.jdbc.JdbcSinkConnector"  
"class":"io.confluent.connect.jdbc.JdbcSourceConnector"  
"class":"org.apache.kafka.connect.mirror.MirrorCheckpointConnector"  
"class":"org.apache.kafka.connect.mirror.MirrorHeartbeatConnector"  
"class":"org.apache.kafka.connect.mirror.MirrorSourceConnector"
```

Lo que hay que mirar. Son cinco. En Confluent Hub hay cientos, o sea S3, Elasticsearch, MongoDB, Salesforce. **Instalar uno es dejar su `.jar` en la carpeta de plugins y reiniciar el *worker*.** No se compila nada.

B · El camino de vuelta, que es el Sink

Es la otra mitad de Connect, y va de Kafka a la base de datos.

```
kafka-cli/verificar-tabla-procesados.sh

curl -s -w "\nHTTP %{http_code}\n" -X POST -H "Content-Type: application/json" \
  --data @infra/connect/jdbc-sink-procesados.json \
  http://localhost:8083/connectors | fold -s -w 88

curl -s http://localhost:8083/connectors/novatech-sink-procesados/status | tr ',' '\n'

kafka-cli/publicar-procesado.sh 6
kafka-cli/verificar-tabla-procesados.sh
```

LO QUE SALE DE LA PRIMERA LÍNEA · LA TABLA ESTÁ VACÍA

```
id | cliente_id | producto | estado | procesado_en
---+-----+-----+-----+-----
(0 rows)
```

Y LO QUE SALE DE LA ÚLTIMA, MEDIDO CUATRO SEGUNDOS DESPUÉS DE PUBLICAR

```
id | cliente_id | producto | estado | procesado_en
---+-----+-----+-----+-----
6 | 1001 | Pedido procesado en 23:50:53 | procesado | 2026-08-29 03:50:55.081539
(1 row)
```

Lo que hay que mirar. Publicaste en un tópico y apareció una fila en SQL. Otra vez sin código, y esta vez en la dirección contraria.

Y dos detalles del JSON del Sink que vale leer en `infra/connect/jdbc-sink-procesados.json`.

Campo	Qué decide
<code>insert.mode: upsert</code> con <code>pk.fields: id</code>	Si publicas dos veces el mismo <code>id</code> , actualiza en vez de duplicar . Es lo que hace que el flujo sea repetible sin ensuciar la tabla
<code>auto.create: false</code>	Connect no crea la tabla. Podría, pero con tipos genéricos. Aquí la creó el <code>init-novatech.sql</code> con los tipos correctos

C · Romper el Sink a propósito, y ver los dos `state` separarse

Publica un mensaje sin el envoltorio que el Sink exige.

```
echo '{"cliente_id":99,"producto":"sin id","cantidad":1,"monto":1.0,"estado":"x"}' \
| docker exec -i kafka-broker-1 kafka-console-producer \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab09.pedidos.procesados

curl -s http://localhost:8083/connectors/novatech-sink-procesados/status | tr ',' '\n'
```

Lo que hay que mirar. El conector sigue en **RUNNING** y el que murió es el **task**, con un **FAILED** y un **trace** larguísimo. La causa raíz está más abajo en el mismo **trace**, y dice que el **JsonConverter** con **schemas.enable** exige los campos **schema** y **payload**.

Y lo que no funciona, que es lo instructivo:

```
curl -s -X POST \
  "http://localhost:8083/connectors/novatech-sink-procesados/restart?
  includeTasks=true&onlyFailed=true"
```

Devuelve **HTTP 202** y el task **vuelve a FAILED**. Es correcto que así sea, porque el mensaje malo sigue en el mismo offset y lo vuelve a leer. **Un restart no arregla un dato malo**. Las salidas de verdad son configurar **errors.tolerance: all** con una cola de mensajes muertos, o **bin/reset-lab.sh**.

D · El importe en base64, y qué se pierde al arreglarlo

```
curl -s -X DELETE -w "HTTP %{http_code}\n" \
  http://localhost:8083/connectors/novatech-source-pedidos
```

LO QUE SALE

HTTP 204

El **204 No Content** es la respuesta correcta a un borrado. Se hizo, y no hay nada que devolver.

Agrega **"numeric.mapping": "best_fit"** al JSON del conector y vuelve a crearlo. El **"ExLQ"** pasa a ser **12500.0**.

LEE ESTO ANTES DE USARLO EN PRODUCCIÓN

best_fit elige el tipo que quepa, y para un decimal grande eso significa un **double**. Es decir, **redondeo**. El base64 era feo y era exacto. **En un sistema tributario, esa fealdad es una característica.**

E · La inserción masiva

```
for i in 1 2 3 4 5 6 7 8 9 10; do
    kafka-cli/insertar-pedido.sh $((3000+i)) "Pedido masivo $i" $i $((10000*i))
done
docker exec kafka-broker-1 kafka-get-offsets \
    --bootstrap-server kafka-broker-1:29092 --topic novatech.lab09.pedidos --time -1
```

Lo que hay que mirar. Las diez filas tardan lo mismo en llegar que **una sola** del paso 5, porque viajaron en la misma pasada. **No es una fila por pasada. Es todo lo que haya bajo el dedo.**

La pregunta que vale. ¿Qué le pasa a este conector el día que alguien haga una carga de dos millones de filas de una vez?

F · Los envoltorios de `connect-cli/`, medidos sin `python3`

El laboratorio trae cinco envoltorios que hacen lo mismo que los `curl` del recorrido, más cortos.

```
connect-cli/list-connectors.sh
connect-cli/status-connector.sh <nombre>
connect-cli/create-source.sh
connect-cli/create-sink.sh
connect-cli/delete-connector.sh <nombre>
```

Funcionan, e imprimen la respuesta ya formateada, que se lee mucho mejor que el `tr ',' '\n'` del recorrido. Úsalos si te sirven y tienes `python3`.

```
python3 --version
grep -c python3 connect-cli/*.sh
```

Ahora, lo que pasa en Git Bash para Windows, que **no trae** `python3`. Esto está medido corriendo los cinco con un `PATH` sin Python, y **no se comportan igual**.

Envoltorio	Qué hace sin <code>python3</code>
<code>delete-connector.sh</code>	Funciona. No usa Python, así que borra y avisa
<code>list-connectors.sh</code>	Funciona. Tiene un <code> </code> de repuesto y te da el JSON sin formatear
<code>status-connector.sh</code>	Imprime su encabezado y nada más . Sin error
<code>create-source.sh</code> y <code>create-sink.sh</code>	La trampa. Ver abajo

LA TRAMPA DE `CREATE-SOURCE.SH`, MEDIDA

Sin `python3`, la salida es ésta y nada más:

```
[Create Source Connector] novatech-source-pedidos
```

Pero el conector sí se creó. El `POST` ya se ejecutó. Lo que murió fue el formateo de la respuesta, y el mensaje de error se lo tragó el `2>/dev/null` del final de la línea.

Así que el alumno cree que falló y lo vuelve a correr. **El segundo intento tampoco dice nada**, y por debajo Connect está contestando un `409` que nadie ve. **Puede pasarse media hora reintentando algo que funcionó a la primera.** Por eso el recorrido va por `curl`, que enseña el código HTTP en la cara.

G · Inspección visual

Kafbat UI, en `http://localhost:8090`. Compara lo que la interfaz muestra con lo que devolvió el `/status`. **Fíjate en si la interfaz distingue los dos `state`**, que es el punto del bloque C.

H · El reporte del laboratorio

`plantillas/reporte-entregable.md` recorre las actividades con sus preguntas. Las respuestas de referencia están en `soluciones/reporte-resuelto.md` y en `soluciones/respuestas-desafio.md`.

Antes de cerrar

DEJA EL TERRENO LIMPIO

```
bin/90-test-lab.sh    # ver el estado del laboratorio
bin/stop-lab.sh      # apagar los contenedores
```

`bin/stop-lab.sh` solo detiene los contenedores y no borra nada. **Ojo al reanudar**, porque `bin/start-lab.sh` reconstruye el laboratorio desde cero y los conectores que creaste no sobreviven. Si modificaste un JSON de conector, guárdalo antes.

LO QUE DICE EL VALIDADOR SI TODO ESTÁ EN ORDEN

Validador del Lab 13 – estado actual

- ✓ los 3 brokers están healthy
- ✓ Kafka Connect responde en :8083/connectors
- ✓ el connector novatech-source-pedidos existe ("state":"RUNNING")

✓ LAB EN BUEN ESTADO (3 verificaciones OK)

LO QUE TE LLEVAS

El programa que no escribiste hoy es el que no vas a tener que mantener el año que viene. **No tiene dueño que se vaya, ni versiones que migrar, ni métricas que inventar.**

A cambio hay que saber dos cosas que hoy viste. **Que los estados son dos**, y que el de arriba puede mentir. **Y que el dedo solo avanza**, así que antes de proponer esto para un caso de SUNAT hay que preguntar si los datos de origen se corrigen después de escritos.